

Integrasi Web API Tingkat Lanjut:

Menguasai Middleware Gin dan Pengenalan GORM

Durasi: 3 Jam (180 Menit) **Level:** Intermediate (Menengah)

Untuk membangun aplikasi web tingkat industri, perutean (*routing*) dan pemrosesan request/response dasar saja tidak cukup. Anda harus mampu mengamankan endpoint API menggunakan **Middleware**, menyaring akses, serta mengelola interaksi database relasional secara lebih efisien menggunakan ORM (Object Relational Mapper) terpopuler di komunitas Go, yaitu **GORM**.

Modul ini akan membedah siklus hidup request di Gin menggunakan Middleware kustom, perbedaan alur kerja `c.Next()` dan `c.Abort()`, cara menghubungkan aplikasi Go ke PostgreSQL via GORM, melakukan migrasi skema otomatis (*AutoMigrate*), serta membangun REST API CRUD yang terintegrasi dengan database secara aman.

Sesi 1: Middleware pada Gin Framework (45 Menit)

Middleware adalah sebuah fungsi perantara yang dieksekusi di tengah-tengah alur request masuk sebelum mencapai handler utama, atau setelah handler utama selesai memproses response. Middleware biasanya digunakan untuk tugas cross-cutting seperti: autentikasi/otorisasi, logging aktivitas request, enkripsi/kompresi, atau pembatasan laju request (*rate limiting*).

1. Struktur Fungsi Middleware

Dalam Gin, middleware dideklarasikan sebagai fungsi yang mengembalikan tipe data `gin.HandlerFunc`.

```
package main

import (
    "fmt"
    "time"

    "github.com/gin-gonic/gin"
)

// Contoh Middleware pencatat waktu eksekusi request
func RequestLogger() gin.HandlerFunc {
```

```

    return func(c *gin.Context) {
        start := time.Now()

        // 1. Meneruskan request ke handler berikutnya (atau handler utama)
        c.Next()

        // 2. Kode di bawah c.Next() akan dieksekusi SETELAH handler utama
        selesai

        latency := time.Since(start)
        status := c.Writer.Status()
        fmt.Printf("[LOGGER] %s | %d | %s\n", c.Request.URL.Path, status,
latency)
    }
}

```

2. Alur Kontrol Context: `c.Next()` VS `c.Abort()`

Di dalam middleware, Anda dapat mengontrol alur eksekusi request menggunakan dua method penting pada `gin.Context` :

- **`c.Next()`** : Menginstruksikan Gin untuk menunda eksekusi sisa kode di middleware saat ini, lalu mengeksekusi handler/middleware berikutnya. Setelah handler berikutnya selesai, sisa eksekusi di middleware ini akan dilanjutkan.
- **`c.Abort()`** : Menghentikan eksekusi handler berikutnya segera. Request akan langsung dihentikan dan tidak akan sampai ke handler utama. Biasanya dipadukan dengan pengiriman error JSON menggunakan `c.AbortWithStatusJSON()` .
- **Contoh Middleware Autentikasi (Token-Based):**

```

func AuthMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        token := c.GetHeader("Authorization")

        // Validasi token statis sederhana
        if token != "Bearer RahasiaNegara123" {
            // Abort request dan kirim HTTP 401 Unauthorized
            c.AbortWithStatusJSON(401, gin.H{
                "error": "Akses ditolak: Token tidak valid atau tidak
disertakan",
            })
            return // Keluar dari middleware agar c.Next() tidak dipanggil
        }

        c.Next()
    }
}

```

3. Pemasangan Middleware

Gin mendukung tiga level pemasangan middleware:

1. **Global Middleware:** Berlaku untuk seluruh rute di server.

```
r := gin.New()
r.Use(RequestLogger()) // Dipasang secara global
```

2. **Route Group Middleware:** Berlaku khusus untuk grup rute tertentu.

```
admin := r.Group("/admin")
admin.Use(AuthMiddleware()) // Hanya berlaku untuk grup /admin
{
    admin.GET("/dashboard", handleDashboard)
}
```

3. **Single Route Middleware:** Berlaku hanya untuk satu endpoint spesifik.

```
r.POST("/checkout", AuthMiddleware(), handleCheckout)
```

Sesi 2: Pengenalan GORM (Object Relational Mapper) (45 Menit)

GORM adalah pustaka ORM ramah developer untuk bahasa Go. GORM menerjemahkan baris data tabel database relasional menjadi struct Go, dan mengubah manipulasi struct tersebut kembali menjadi perintah kueri SQL secara otomatis.

1. Instalasi GORM dan Driver PostgreSQL

Jalankan perintah ini di terminal proyek Anda untuk menginstal paket core GORM beserta driver database PostgreSQL:

```
go get -u gorm.io/gorm
go get -u gorm.io/driver/postgres
```

2. Membuat Koneksi Database GORM

Koneksi database dibangun sekali saat inisialisasi aplikasi menggunakan `gorm.Open`.

```
package main

import (
```

```

    "fmt"
    "log"

    "gorm.io/driver/postgres"
    "gorm.io/gorm"
)

func InitDB() *gorm.DB {
    dsn := "host=localhost user=postgres password=secret45 dbname=ecommercedb
port=5434 sslmode=disable TimeZone=Asia/Jakarta"
    db, err := gorm.Open(postgres.Open(dsn), &gorm.Config{})
    if err != nil {
        log.Fatalf("Gagal terhubung ke database via GORM: %v", err)
    }

    fmt.Println("Koneksi database GORM berhasil didirikan!")
    return db
}

```

3. Pemodelan Struct & Auto-Migration

GORM memetakan tipe struct menjadi tabel database. Secara default, GORM menggunakan penamaan kolom *snake_case* (contoh: properti `CreatedBy` dipetakan menjadi kolom `created_by`).

- **Auto-Migration:** Fitur untuk membuat atau memperbarui tabel database secara otomatis berdasarkan definisi struct Go.

```

type Product struct {
    ID          uint          `gorm:"primaryKey;autoIncrement"`
    SKU         string        `gorm:"type:varchar(50);uniqueIndex;not null"`
    Name        string        `gorm:"type:varchar(100);not null"`
    Price       float64       `gorm:"type:numeric(12,2);not null"`
    Stock       int           `gorm:"type:int;default:0"`
    CreatedAt   time.Time     // Otomatis diisi waktu buat
    UpdatedAt   time.Time     // Otomatis diperbarui saat update
    DeletedAt   gorm.DeletedAt `gorm:"index"` -- Mengaktifkan fitur Soft Delete
}

```

NOTE

Soft Delete: Jika struct model memiliki kolom `DeletedAt` bertipe `gorm.DeletedAt`, pemanggilan perintah `db.Delete()` tidak akan menghapus baris data secara fisik dari tabel. GORM hanya akan mengisi kolom `deleted_at` dengan timestamp waktu penghapusan. Query `SELECT` biasa otomatis akan mengabaikan data yang sudah dihapus secara lunak ini.

Sesi 3: Operasi CRUD Dasar dengan GORM (45 Menit)

Setelah mendirikan koneksi dan menguji model, kita dapat melakukan manipulasi data relasional dengan sintaks Go yang bersih.

1. Create (Menyimpan Data Baru)

```
newProd := Product{
    SKU:    "SKU-EL-99",
    Name:   "Keyboard RGB Mechanical",
    Price:  850000.00,
    Stock:  20,
}
result := db.Create(&newProd)
// result.Error berisi error database jika ada (misal unique constraint violation)
// result.RowsAffected mengembalikan jumlah baris terpengaruh (1)
```

2. Read (Membaca Data)

- Mengambil data berdasarkan ID (Primary Key):

```
var p Product
db.First(&p, 1) // SELECT * FROM products WHERE id = 1 LIMIT 1
```

- Mengambil semua data dengan kondisi filter:

```
var list []Product
db.Where("price > ?", 500000.00).Find(&list)
```

- Pencarian aman yang menangani error "Record Not Found":

```
err := db.First(&p, "sku = ?", "SKU-TIDAK-ADA").Error
if errors.Is(err, gorm.ErrRecordNotFound) {
    fmt.Println("Barang tidak ada di database.")
}
```

3. Update (Mengubah Data)

Saat melakukan pembaruan data menggunakan struct, GORM **hanya akan meng-update kolom yang memiliki nilai non-zero** (tidak nol/bukan nilai default tipe data).

- Update Kolom Tertentu (Aman untuk Nilai Nol/Default):

```
// Mengubah stok menjadi 0 dan memperbarui harga produk ID 1
db.Model(&Product{}).Where("id = ?", 1).Updates(map[string]interface{}{
    "stock": 0,
```

```
"price": 750000.00,  
})
```

- Update Seluruh Kolom Struct (Termasuk Nol/Default):

```
// Gunakan Select("*") untuk memaksa GORM menulis semua nilai field struct  
p.Stock = 0  
p.Price = 800000.00  
db.Model(&p).Select("*").Updates(&p)
```

4. Delete (Menghapus Data)

- Soft Delete (Lunak):

```
db.Delete(&Product{}, 1) // Mengisi kolom deleted_at produk ID 1
```

- Hard Delete (Fisik/Permanen):

```
db.Unscoped().Delete(&Product{}, 1) // Menghapus baris permanen dari disk
```

Sesi 4: Praktikum Mandiri: API E-Commerce Aman dengan Gin, GORM, dan Middleware Auth (45 Menit)

Kita akan mengintegrasikan Gin Web Framework dengan GORM dan database PostgreSQL nyata (ecommercedb port 5434). Kita juga akan mengamankan endpoint CRUD produk menggunakan Middleware Autentikasi Token.

Implementasi Kode (main.go)

```
package main  
  
import (  
    "errors"  
    "net/http"  
    "strconv"  
    "time"  
  
    "github.com/gin-gonic/gin"  
    "gorm.io/driver/postgres"  
    "gorm.io/gorm"  
)  
  
// 1. Struct Model GORM
```

```

type Product struct {
    ID      uint      `gorm:"primaryKey;autoIncrement" json:"id"`
    SKU     string     `gorm:"type:varchar(50);uniqueIndex;not null"
    json:"sku" binding:"required"`
    Name    string     `gorm:"type:varchar(100);not null" json:"name"
    binding:"required"`
    Price   float64    `gorm:"type:numeric(12,2);not null" json:"price"
    binding:"required,gt=0"`
    Stock   int       `gorm:"type:int;default:0" json:"stock"
    binding:"required,min=0"`
    CreatedAt time.Time `json:"created_at"`
    UpdatedAt time.Time `json:"updated_at"`
    DeletedAt gorm.DeletedAt `gorm:"index" json:"- "` // Disembunyikan dari JSON
    response
}

var db *gorm.DB

// 2. Inisialisasi Database
func initDatabase() {
    dsn := "host=localhost user=postgres password=secret45 dbname=ecommercedb
port=5434 sslmode=disable TimeZone=Asia/Jakarta"
    var err error
    db, err = gorm.Open(postgres.Open(dsn), &gorm.Config{})
    if err != nil {
        panic("Gagal terkoneksi ke database: " + err.Error())
    }

    // Jalankan Auto Migration
    _ = db.AutoMigrate(&Product{})
}

// 3. Custom Auth Middleware
func ApiKeyAuth() gin.HandlerFunc {
    return func(c *gin.Context) {
        token := c.GetHeader("X-API-KEY")
        if token != "SUPER-SECRET-KEY-99" {
            c.AbortWithStatusJSON(http.StatusUnauthorized, gin.H{
                "error": "Akses tidak sah! X-API-KEY tidak valid.",
            })
            return
        }
        c.Next()
    }
}

func main() {
    // Inisialisasi DB
    initDatabase()

    r := gin.Default()

    // Route Group dengan Proteksi Middleware

```

```

v1 := r.Group("/api/v1")
v1.Use(ApiKeyAuth()) // Semua rute di dalam v1 wajib melewati autentikasi X-
API-KEY
{
    // GET /api/v1/products - Mendapatkan semua produk
    v1.GET("/products", func(c *gin.Context) {
        var list []Product
        search := c.Query("search")

        query := db
        if search != "" {
            query = query.Where("name ILIKE ?", "%"+search+"%")
        }

        if err := query.Find(&list).Error; err != nil {
            c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})

            return
        }
        c.JSON(http.StatusOK, gin.H{"data": list})
    })

    // GET /api/v1/products/:id - Detail Produk
    v1.GET("/products/:id", func(c *gin.Context) {
        idStr := c.Param("id")
        id, _ := strconv.Atoi(idStr)

        var p Product
        if err := db.First(&p, id).Error; err != nil {
            if errors.Is(err, gorm.ErrRecordNotFound) {
                c.JSON(http.StatusNotFound, gin.H{"error":
"Produk tidak ditemukan"})

                return
            }
            c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})

            return
        }

        c.JSON(http.StatusOK, gin.H{"data": p})
    })

    // POST /api/v1/products - Buat Produk Baru
    v1.POST("/products", func(c *gin.Context) {
        var input Product
        if err := c.ShouldBindJSON(&input); err != nil {
            c.JSON(http.StatusBadRequest, gin.H{"error":
err.Error()})

            return
        }

        // Simpan ke database via GORM
        if err := db.Create(&input).Error; err != nil {

```



```

        c.JSON(http.StatusConflict, gin.H{"error": "Gagal
membuat produk. SKU kemungkinan sudah ada."})
        return
    }

    c.JSON(http.StatusCreated, gin.H{
        "message": "Produk berhasil ditambahkan ke database!",
        "data":    input,
    })
})

// PUT /api/v1/products/:id - Update Produk
v1.PUT("/products/:id", func(c *gin.Context) {
    idStr := c.Param("id")
    id, _ := strconv.Atoi(idStr)

    var p Product
    if err := db.First(&p, id).Error; err != nil {
        if errors.Is(err, gorm.ErrRecordNotFound) {
            c.JSON(http.StatusNotFound, gin.H{"error":
"Produk tidak ditemukan"})
            return
        }
        c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})
        return
    }

    var input Product
    if err := c.ShouldBindJSON(&input); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{"error":
err.Error()})
        return
    }

    // Lakukan pembaruan kolom secara eksplisit menggunakan map
    // Agar nilai 0 pada stock atau price tetap dapat diperbarui
    updateData := map[string]interface{}{
        "sku":    input.SKU,
        "name":   input.Name,
        "price":  input.Price,
        "stock":  input.Stock,
    }

    if err := db.Model(&p).Updates(updateData).Error; err != nil {
        c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})
        return
    }

    c.JSON(http.StatusOK, gin.H{
        "message": "Produk berhasil diperbarui!",
        "data":    p,
    })
})

```

```

    })

    // DELETE /api/v1/products/:id - Soft Delete Produk
    v1.DELETE("/products/:id", func(c *gin.Context) {
        idStr := c.Param("id")
        id, _ := strconv.Atoi(idStr)

        var p Product
        if err := db.First(&p, id).Error; err != nil {
            if errors.Is(err, gorm.ErrRecordNotFound) {
                c.JSON(http.StatusNotFound, gin.H{"error":
"Produk tidak ditemukan"})

                return
            }
            c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})

            return
        }

        // Melakukan Soft Delete
        if err := db.Delete(&p).Error; err != nil {
            c.JSON(http.StatusInternalServerError, gin.H{"error":
err.Error()})

            return
        }

        c.JSON(http.StatusOK, gin.H{"message": "Produk berhasil
dihapus secara lunak (Soft Deleted)"})
    })
}

r.Run(":8080")
}

```

Sesi 5: Soal Latihan Praktik E-Commerce (Mandiri)

Selesaikan dua latihan pemeliharaan API dan database di bawah ini untuk memperkuat pemahaman Anda mengenai integrasi middleware kustom dan operasi relasional GORM.

Latihan 1: Middleware Log Waktu Eksekusi (Execution Time Logger) & Rate Limiter Sederhana

Deskripsi Soal: Dalam lingkungan backend produksi, penting untuk memantau performa kecepatan endpoint dan mencegah penyalahgunaan API dari klien luar (*abuse request*).

Kriteria Implementasi:

1. Buatlah sebuah custom middleware Gin bernama `SimpleRateLimiter` .
2. Di dalam middleware tersebut, simpan daftar log jumlah hit dari setiap alamat IP klien (menggunakan map in-memory sederhana `map[string]int` yang menyimpan IP sebagai kunci dan jumlah hit sebagai nilai).

TIP

Dapatkan alamat IP klien melalui properti konteks `c.ClientIP()` .

3. Batasi bahwa setiap IP hanya diperbolehkan mengirim request maksimal **10 kali** ke server. Jika lebih dari 10, hentikan request dengan `c.AbortWithStatusJSON` status 429 Too Many Requests beserta pesan error yang informatif.
4. Daftarkan middleware `SimpleRateLimiter` ini secara global di dalam fungsi `main()` .

Latihan 2: Pembuatan Relasi One-to-Many pada GORM (Kategori & Produk)

Deskripsi Soal: Sejauh ini model `Product` belum terhubung secara struktural dengan model `Category` di tingkat kode GORM. Hubungkan kedua entitas tersebut menggunakan deklarasi relasi asosiasi GORM.

Kriteria Implementasi:

1. Buatlah struct model `Category` dengan properti:
 - `ID` (uint - Primary Key)
 - `Name` (string - Unique, Not Null)
 - `Products` ([]Product - Mengindikasikan relasi Has Many)
2. Perbarui struct model `Product` agar memiliki kolom Foreign Key:
 - `CategoryID` (uint - Foreign Key)
 - `Category` (Category - struct relasi Belongs To dengan tag `gorm:"foreignKey:CategoryID"`)
3. Jalankan `db.AutoMigrate(&Category{}, &Product{})` di bagian inisialisasi database.
4. Buatlah satu endpoint baru `GET /api/v1/categories/:id/products` untuk menampilkan daftar produk yang berada di bawah kategori spesifik. Gunakan fitur GORM **Preloading** (`db.Preload("Products")`) untuk mengambil data kategori beserta relasi daftar produknya sekaligus dalam satu kueri efisien.